Experiment No. 4

Name: Parth Vijay Gulhane
Roll No: 23CO043
Class: TE Comp A

Title:- Implement a solution for a Constraint Satisfaction Problem using Branch and Bound and Backtracking for nqueens problem or a graph coloring problem.

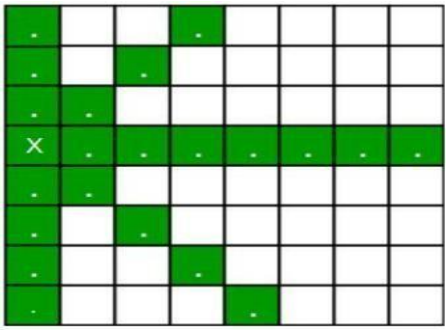
Objectives:- To study SQL DML statements

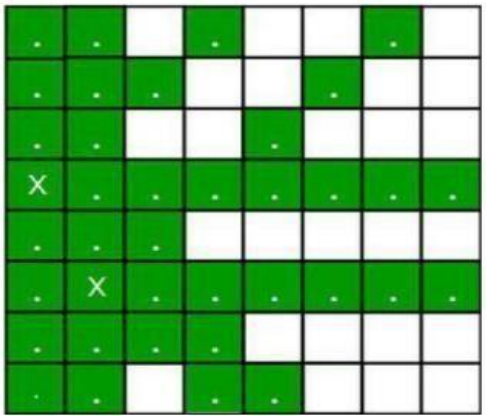
THEORY: Queen Problem using Branch and Bound

The N queens puzzle is the problem of placing N chess queens on an N×N chessboard so that no two queens threaten each other. Thus, a solution requires that no two queens share the same row, column, or diagonal.

The backtracking Algorithm for N-Queen is already discussed here. In a backtracking solution, we backtrack when we hit a dead end. In Branch and Bound solution, after building a partial solution, we figure out that there is no point going any deeper as we are going to hit a dead end.

Let's begin by describing the backtracking solution. The idea is to place queens one by one in different columns, starting from the leftmost column. When we place a queen in a column, we check for clashes with already placed queens. In the current column, if we find a row for which there is no clash, we mark this row and column as part of the solution. If we do not find such a row due to clashes, then we backtrack and return false.





Placing 1st queen on (3, 0) and 2nd queen on (5, 1)

For the 1st Queen, there are total 8 possibilities as we can place 1st Queen in any row of first column. Let's place Queen 1 on row 3. After placing 1st Queen, there are 7 possibilities left for the 2nd Queen . But wait, we don't really have 7 possibilities. We cannot place Queen 2 on rows 2, 3 or 4 as those cells are under attack from Queen 1. So, Queen 2 has only 8 – 3 = 5 valid positions left.

After picking a position for Queen 2, Queen 3 has even fewer options as most of the cells in its column are under attack from the first 2 Queens.

We need to figure out an efficient way of keeping track of which cells are under attack. In previous solution we kept an 8-by8 Boolean matrix and update it each time we placed a queen, but that required linear time to update as we need to check for safe cells.

Basically, we have to ensure 4 things:

1. No two queens share a column.

2. No two queens share a row.

3. No two queens share a top-right to left-bottom diagonal.

4. No two queens share a top-left to bottom-right diagonal.

Number 1 is automatic because of the way we store the solution. For number 2, 3 and 4, we can perform updates in O (1) time. The idea is to keep three Boolean arrays that tell us which rows and which diagonals are occupied.

Let’s do some pre-processing first. Let9s create two N x N matrix one for / diagonal and other one for \ diagonal. Let’s call them slashCode and backslashCode respectively. The trick is to fill them in such a way that two queens sharing a same /diagonal will have the same value in matrix slashCode, and if they share same \- diagonal, they will have the same value in backslashCode matrix.

For an N x N matrix, fill slashCode and backslashCode matrix using below formula –

slashCode[row][col]= row+col

backslashCode[row][col]= row – col +(N-1)

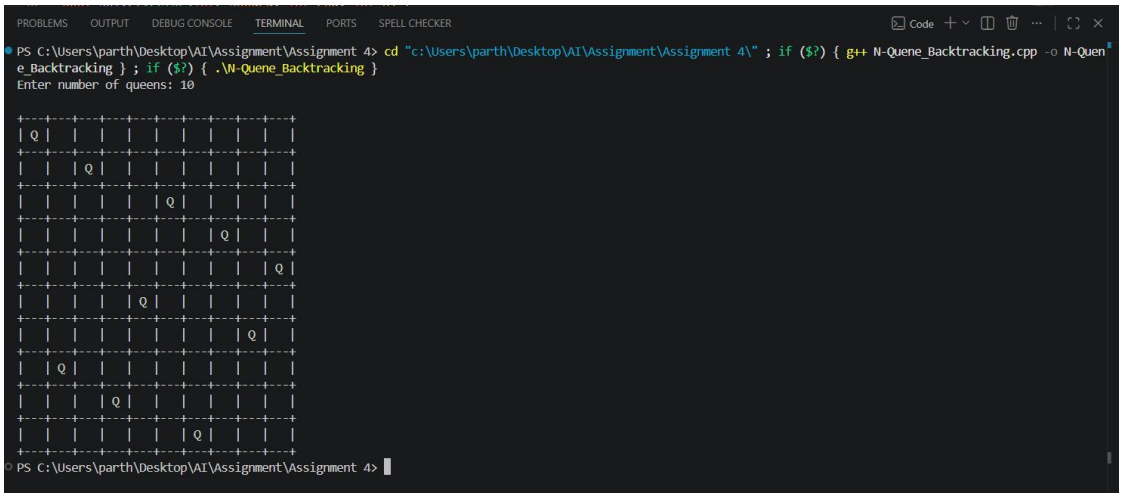
Using above formula will result in below matrices

N-Queens (Backtracking) with Grid Output Code

```
#include <iostream>
#include <vector>
#include <cmath>
using namespace std;

// Check if safe to place queen
bool is_safe(vector<int> &board, int row, int col, int n) {
    for (int i = 0; i < row; i++) {
        // Same column
        if (board[i] == col)
            return false;
        // Diagonal check
        if (abs(board[i] - col) == abs(i - row))
            return false;
    }
    return true;
}
// Print board
void print_board(vector<int> &board, int n) {
    cout << endl;
    for (int i = 0; i < n; i++) {
        // Horizontal line
        for (int j = 0; j < n; j++)
            cout << "+---";
        cout << "+" << endl;
        // Row with queen
        for (int j = 0; j < n; j++) {
            if (board[i] == j)
                cout << "| Q ";
            else
                cout << "|   ";
        }
        cout << "|" << endl;
    }
    // Bottom line
    for (int j = 0; j < n; j++)
        cout << "+---";
    cout << "+" << endl;
}
// Solve using backtracking
bool solve(vector<int> &board, int row, int n) {
    if (row == n) {
        print_board(board, n);
        return true; // stop after first solution
    }
```

```
    for (int col = 0; col < n; col++) {
        if (is_safe(board, row, col, n)) {
            board[row] = col;
            if (solve(board, row + 1, n))
                return true;
            // Backtrack
            board[row] = -1;
        }
    }
    return false;
}

// Main function
void solve_n_queens(int n) {
    vector<int> board(n, -1);
    if (!solve(board, 0, n)) {
        cout << "No solution exists" << endl;
    }
}

int main() {
    int n;
    cout << "Enter number of queens: ";
    cin >> n;
    solve_n_queens(n);
    return 0;
}
```

Output



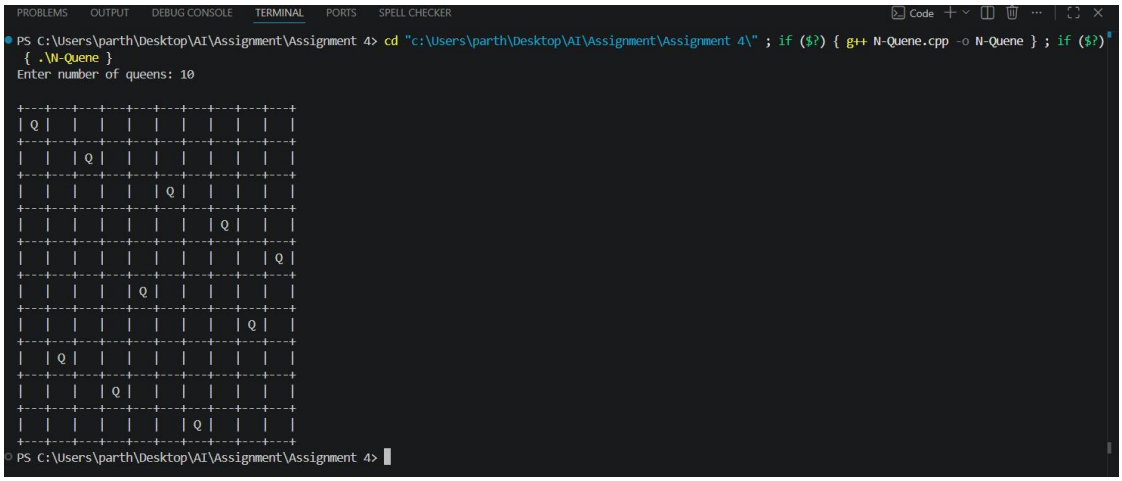
N-Queens using Branch & Bound (with grid output)

Code:

```
#include <iostream>
#include <vector>
using namespace std;

// Print board
void print_board(vector<vector<int>> &board, int n) {
    cout << endl;
    for (int i = 0; i < n; i++) {
        // Top border
        for (int j = 0; j < n; j++)
            cout << "+---";
        cout << "+" << endl;
        // Row content
        for (int j = 0; j < n; j++) {
            if (board[i][j] == 1)
                cout << "| Q ";
            else
                cout << "|   ";
        }
        cout << "|" << endl;
    }
    // Bottom border
    for (int j = 0; j < n; j++)
        cout << "+---";
    cout << "+" << endl;
}
// Solve using Branch & Bound
bool solve(int row,
           vector<vector<int>> &board,
           vector<bool> &col,
           vector<bool> &diag1,
           vector<bool> &diag2,
           int n) {
    if (row == n) {
        print_board(board, n);
        return true; // stop after first solution
    }
    for (int c = 0; c < n; c++) {
        // Check safety using bounding arrays
```

```
        if (!col[c] &&
            !diag1[row - c + n - 1] &&
            !diag2[row + c]) {
            // Place queen
            board[row][c] = 1;
            col[c] = true;
            diag1[row - c + n - 1] = true;
            diag2[row + c] = true;
            if (solve(row + 1, board, col, diag1, diag2, n))
                return true;
            // Backtrack
            board[row][c] = 0;
            col[c] = false;
            diag1[row - c + n - 1] = false;
            diag2[row + c] = false;
        }
    }
    return false;
}
// Main function
void solve_n_queens_bb(int n) {
    vector<vector<int>> board(n, vector<int>(n, 0));
    vector<bool> col(n, false);
    vector<bool> diag1(2*n - 1, false);
    vector<bool> diag2(2*n - 1, false);
    if (!solve(0, board, col, diag1, diag2, n)) {
        cout << "No solution exists" << endl;
    }
}

int main() {
    int n;
    cout << "Enter number of queens: ";
    cin >> n;
    solve_n_queens_bb(n);
    return 0;
}
```

Output:



Conclusion: Thus we have studied backtracking and branch and bound. Students tried to understand when these algorithmic approaches can be applied to problems of interest.